How-To Library

Informatica

Configuring Secure Communication to Microsoft SQL Server in PowerCenter

© Copyright Informatica LLC 2016, 2021. Informatica Corporation. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica Corporation. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

You can enable secure communication with SSL encryption from PowerCenter to Microsoft SQL Server. When you use an ODBC provider type in the Microsoft SQL Server connection, you must enable DSN to set the SSL encryption. This article describes how you can configure secure communication from PowerCenter to Microsoft SQL Server with a native connection for Windows and UNIX.

Supported Versions

- PowerCenter 10.0 - 10.1.1

Table of Contents

Overview. 2
Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore 2
Step 2. Configure a Microsoft SQL Server Data Source 3
Configure a Microsoft SQL Server Data Source for Windows. 3
Configure a Microsoft SQL Server Data Source on UNIX. 4
Step 3. Test a Microsoft SQL Server Connection to Microsoft SQL Server 5
Test a Microsoft SQL Server Connection to Microsoft SQL Server on Windows. 5
Test a Microsoft SQL Server Connection to Microsoft SQL Server on UNIX. 6
Step 4. Configure a Microsoft SQL Server Connection in PowerCenter. 6

Overview

You can enable secure communication from PowerCenter to Microsoft SQL Server. When you read or write data to a third-party database, you can secure the communication with SSL encryption provided by the third-party database. In PowerCenter, you can configure secure communication to Microsoft SQL Server when you run a mapping with Microsoft SQL Server sources or targets.

To configure secure communication to Microsoft SQL Server on the machine where you run the PowerCenter Integration Service, perform the following tasks:

1. Import a Microsoft SQL Server SSL certificate to a truststore.
2. Configure a Microsoft SQL Server data source.
3. Test a Microsoft SQL Server connection to Microsoft SQL Server.
4. Create the Microsoft SQL Server connection in PowerCenter and run the mapping.

Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore

To authenticate the Microsoft SQL Server SSL certificate, you must import the Microsoft SQL Server's public certificate into the client's truststore.

Before you authenticate the Microsoft SQL Server SSL certificate, you must install and download OpenSSL.

You can download OpenSSL at the following link: https://www.openssl.org/community/binaries.html

1. Ask your database administrator to provide you with the Microsoft SQL Server SSL certificate. For example, your database administrator can provide you with the certificate file, `mysqlserver.cer`.
2. Go to **Start** > **Run**.
3. In the Run dialog box, enter `cmd`.
4. At the command prompt, enter the `openssl` command to start OpenSSL.
5. Based on the server certificate format, run the appropriate openssl command to generate an output text.
   - If the server certificate is in PEM format, run the following openssl command:
     `x509 -inform PEM -in c:\mysqlserver.cer -text`
   - If the server certificate is in DER format, run the following openssl command:
     `x509 -inform DER -in c:\mysqlserver.cer -text`
6. Copy the output text and save it as a .txt file. For example, `mysqlserver.txt`
7. At the OpenSSL prompt, enter the following command to generate the truststore file (*.pfx) from the server SSL certificate:

   ```
   pkcs12 -in <fullpath_to_your_server_SSL_certificate_output_text> -out
   <full_path_to_truststore_filename.pfx> -passout <pass_phrase_to_encrypt_output_private_keys>
   -nokeys -export
   ```

   For example, enter the following command:

   ```
   pkcs12 -in C:\temp\mysqlserver.txt
   -out C:\temp1\TrustStore.pfx -passout pass: -nokeys -export
   ```

   where `mysqlserver.txt` is the output that you generated in the previous command. TrustStore is the name of the truststore file that you plan to generate.

The command creates the PKCS#12 file by reading the certificates in the input path. The command specifies the output file name to write the PKCS#12 file without writing any private keys in the output file.

Step 2. Configure a Microsoft SQL Server Data Source

Effective in PowerCenter 10.0, the Microsoft SQL Server native connection uses DataDirect SQL Server ODBC driver. You can configure the Microsoft SQL Server data source in different ways based on whether the PowerCenter Integration Service runs on Windows or UNIX.

Configure a Microsoft SQL Server Data Source for Windows

1. Open the Microsoft ODBC Administrator.
2. Go to the system DSN tab.
3. Click **Add**.
4. Select **DataDirect 7.1 New SQL Server Wire Protocol**.
5. Click **Finish**.
6. Enter the data source name.
7. Enter the host name.

   A host name is the machine where you installed the Microsoft SQL Server database.
8. Enter the port number.

   Default is 1433.
9. Enter the Microsoft SQL Server database name.

10. On the security tab of DSN, set the following secure database parameters:

Property	Description
Encryption Method	Required. Indicates whether data is encrypted when transmitted over the network. This parameter must be set to SSL. For example: `Encryption Method = 1 - SSL`
ValidateServerCertificate	Required. Indicates whether Informatica validates the certificate that the database server sends. Default is true. **Note:** To validate a server certificate, you must use a CA-signed SSL certificate. If you are using a self-signed SSL certificate, disable validate server certificate. Otherwise, the connection will fail.
HostName in Certificate	Required if ValidateServerCertificate is true. Host name of the machine that hosts the secure database. If you specify a host name, PowerCenter validates the host name included in the connection string against the host name in the SSL certificate. You can verify the host name with the help of your database administrator. The host name is case sensitive.
CryptoProtocolVersion	Optional. Specifies the cryptographic protocol to use to connect to a secure database. Set the value based on the database settings recommended by your database administrator.
Truststore	Required if ValidateServerCertificate is true. Absolute path of the truststore file name that you created in "Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore " on page 2.
Truststore Password	Required if ValidateServerCertificate is true. Password that you used in "Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore " on page 2.

Configure a Microsoft SQL Server Data Source on UNIX

1. Set the environment variable ODBCINI to point it to the odbc.ini file path.

   By default, the odbc.ini path is in the following location: `<INFA_HOME>\ODBC7.1\odbc.ini`.
2. Open the odbc.ini file.
3. Under the ODBC Data Sources section of the odbc.ini file, add a data source name with a description.

   For example, you can enter the following data source details in the odbc.ini file:

```
[ODBC Data Sources]
    SQLSERVER_SSL=SQL Server with SSL encryption
```

4. Enter the following properties for the new data source entry in the odbc.ini file:

Property	Description
Driver	Required. Absolute path to the Data Direct New SQL Server ODBC driver. By default, the driver is located at `<INFA_HOME>/ODBC7.1/lib/DWsqls27.so`
Description	Optional. Describes the connection information.
HostName	Required. Name of the machine where you installed the Microsoft SQL Server database.

Property	Description
PortNumber	Required. Port where the Microsoft SQL Server database server listens. Default is 1433.
Database	Required. Name of the Microsoft SQL Server database.
EncryptionMethod	Required. Indicates whether data is encrypted when transmitted over the network. This parameter must be set to SSL. To enable SSL, set it to 1. To disable, set it to 0. Default is disabled.
ValidateServerCertificate	Required. Indicates whether Informatica validates the certificate that the database server sends. To enable, set it to 1. To disable, set it to 0. Default is 1 (Enabled). **Note:** Validate server certificate will work only if you are using a CA-signed certificate. If you are using a self-signed certificate, disable validate server certificate. Else, the connection will fail.
TrustStore	Required if ValidateServerCertificate is true. Absolute path of the truststore file name that you earlier created in "Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore " on page 2.
TrustStore Password	Required if ValidateServerCertificate is true. Password that you used earlier in "Step 1. Import a Microsoft SQL Server SSL Certificate to a Truststore " on page 2.
HostNameInCertificate	Required if ValidateServerCertificate is true. Host name of the machine that hosts the secure database. If you specify a host name, Informatica validates the host name included in the connection string against the host name in the SSL certificate. You can verify the host name with the help of your database administrator. The host name is case-sensitive.
CryptoProtocolVersion	Optional. Specifies the cryptographic protocol to use to connect to a secure database. Set the value based on the database settings recommended by your database administrator.

Sample data source entry:

```
[SQLSERVER_SSL]
    Driver=/home/Informatica/10.0.0/ODBC7.1/lib/DWsqls27.so
    Description=SQL Server Connection with encryption
    HostName=INQAKRH01
    PortNumber=1433
    Database=SQLDB
    EncryptionMethod=1
    ValidateServerCertificate=1
    TrustStore=/home/truststore/TrustStore_INQAKRH01.pfx
    TrustStorePassword=Trustpass123
    HostNameInCertificate=inqakrh01.informatica.com
    CryptoProtocolVersion=TLSv1.2
```

Step 3. Test a Microsoft SQL Server Connection to Microsoft SQL Server

You can test the Microsoft SQL Server connection in different ways depending on whether the PowerCenter Integration Service runs on Windows or UNIX.

Test a Microsoft SQL Server Connection to Microsoft SQL Server on Windows

1. On Windows, open Microsoft ODBC Administrator.
2. Select the DSN that you created in "Configure a Microsoft SQL Server Data Source for Windows" on page 3.

3. Click **Configure**.
4. Click **Test Connect**.
5. Enter the valid database user name and password.
6. Click **OK**.

Test a Microsoft SQL Server Connection to Microsoft SQL Server on UNIX

To test connection on UNIX, use the Informatica Global Customer Support toolssgodbc present under the debugtools folder of the Informatica installation directory. Ensure that you have configured the ODBCINI environment variable to use the ssgodbc command.

1. In the command prompt, navigate to the location of the ssgodbc file path.

   The ssgodbc file is in the following directory:

   ```
   <INFA_HOME>/tools/debugtools/ssgodbc/<linux or unix version>
   ```

   For example, you can find the ssgodbc command for Linux 64 bit in the following path: `<INFA_HOME>/tools/debugtools/ssgodbc/linux64/ssgodbc.linux64`
2. Run the ssgodbc command.

   For example, enter the following ssgodbc command:

   ```
   ssgodbc.linux64 -d SQLSERVER_SSL -u sqluser -p sqlpass123 -v
   ```

   In the example, -d refers to the data source name, -u refers to the database user name, -p refers to the database password, and -v refers to the verbose output.

If the test connection is successful, the command prompt displays the database version and other details. You can close the ssgodbc command manually by pressing CTRL + C.

If the test connection fails, you can review the related error message and edit the connection.

Step 4. Configure a Microsoft SQL Server Connection in PowerCenter

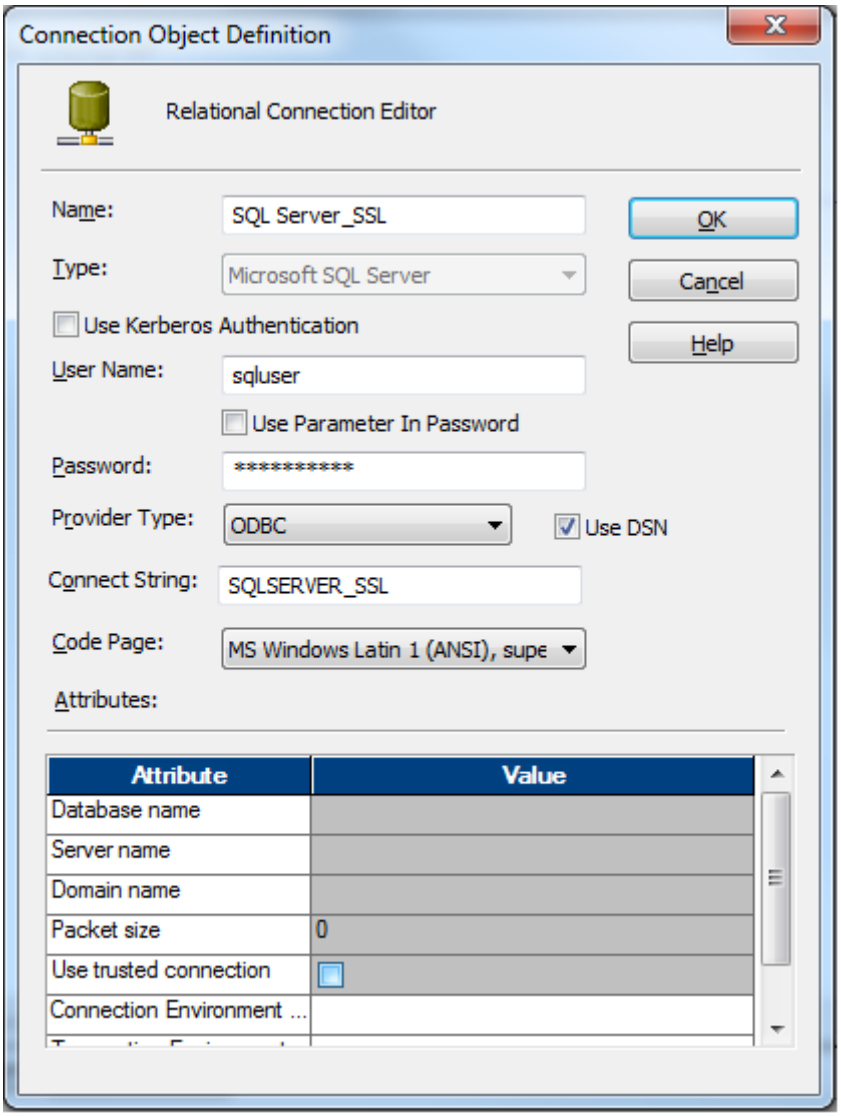
You can configure the connection to Microsoft SQL Server from the Workflow Manager.

1. Log in to the Workflow Manager.
2. Click **Connections** > **Relational**.

   The **Relational Connection Browser** dialog box appears.
3. Click **New**.

   The **Select Type** dialog box appears.
4. Select **Microsoft SQL Server** from the **Select Type** list.
5. Click **New** to create the Microsoft SQL Server connection.
6. Click **OK**.

The **Connection Object Definition** dialog box appears.



7. Enter the connection properties.
8. Select the **Use DSN** option.

   Verify that the connection string is the data source name that you created, such as SQLSERVER_SSL.
9. Click **OK**.

   The database connection appears in the **Relational Connection Browser** list.

Select the Microsoft SQL Server connection to run the mapping with secure communication in PowerCenter.

Author

Sujitha Alexander